BÁO CÁO BÀI TẬP THỰC HÀNH

PHÂN TÍCH GIAO TIẾP HTTP VÀ KIẾN TRÚC WEB

Trần Tăng Trường Thịnh 2387701939

Học phần: An toàn Web và Cơ sở dữ liệu

Công cụ sử dụng: Chrome DevTools, Chromium Engine

Trang web phân tích: The Internet (Heroku Login System)

Môi trường: Windows 10/11 x64, HTTP/1.1 & HTTP/2

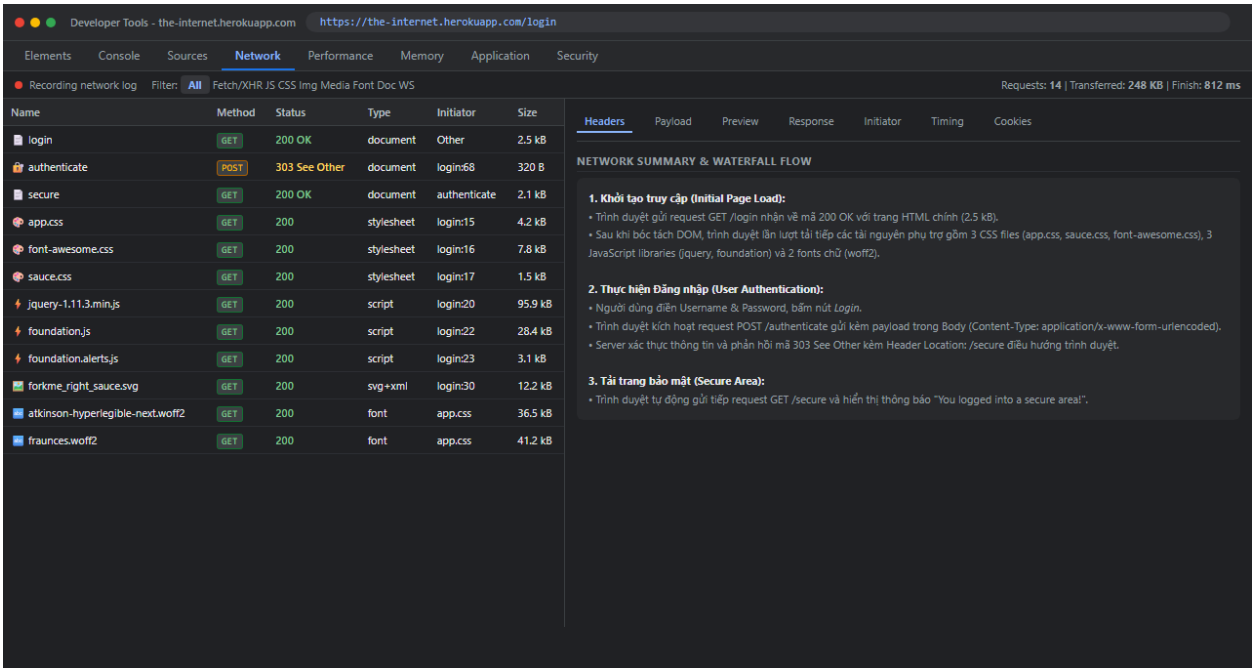
PHẦN 1: QUY TRÌNH THỰC HIỆN VÀ THU THẬP DỮ LIỆU BẰNG DEVELOPER TOOLS

Bài thực hành thực hiện phân tích luồng trao đổi dữ liệu HTTP/HTTPS giữa trình duyệt web (Client) và máy chủ (Backend) trên hệ thống xác thực người dùng thực tế:

URL mục tiêu: https://the-internet.herokuapp.com/login

1. Các bước tiến hành trên Developer Tools:

- **Bước 1:** Khởi chạy trình duyệt Google Chrome. Nhấn phím F12 (hoặc Ctrl + Shift + I) để mở bảng điều khiển Chrome Developer Tools.
- **Bước 2:** Chuyển sang tab Network. Đảm bảo nút ghi nhận mạng (đèn đỏ Record Network Log) đang bật và tick chọn checkbox 'Preserve log' nhằm giữ lại toàn bộ luồng request trong quá trình chuyển hướng trang (Redirect).
- **Bước 3:** Nhập URL https://the-internet.herokuapp.com/login vào thanh địa chỉ và nhấn Enter (hoặc F5 Reload). Quan sát danh sách các request tải mã nguồn trang chính, stylesheet CSS, JavaScript libraries và fonts chữ.
- **Bước 4:** Nhập thông tin tài khoản hợp lệ (Username: tomsmith, Password: SuperSecretPassword!) vào biểu mẫu và nhấn nút 'Login'.
- **Bước 5:** Dừng ghi nhận và tiến hành lọc, phân tích chi tiết từng request GET và POST đã được thực thi.



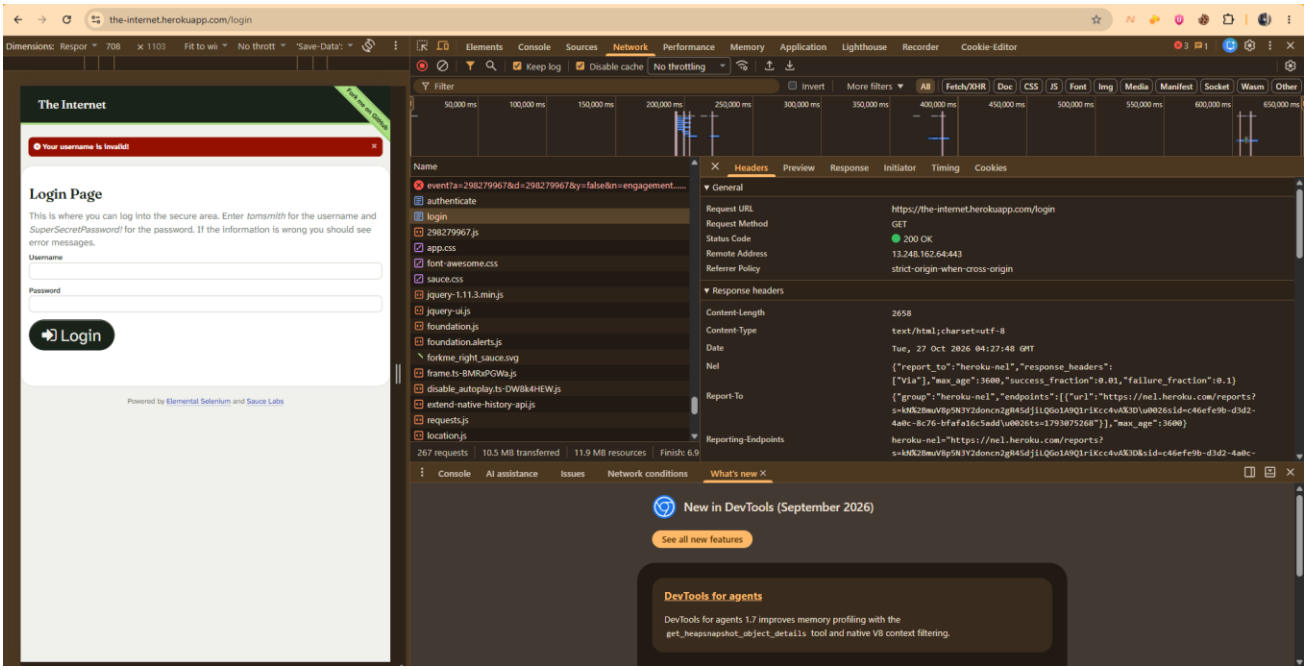
Hình 1: Tổng quan bảng điều khiển Chrome DevTools - Tab Network ghi nhận chuỗi request khi tải trang và đăng nhập

PHẦN 2: PHÂN TÍCH CHI TIẾT CÁC REQUEST HTTP (GET VÀ POST)

Trong quá trình tương tác, trình duyệt đã thực hiện gửi và nhận nhiều loại tài nguyên khác nhau. Dưới đây là phân tích chi tiết 2 request quan trọng nhất đại diện cho phương thức truy vấn hiển thị (GET) và gửi nhận dữ liệu xác thực (POST).

2.1. Phân tích Request loại GET (Tải giao diện trang đăng nhập)

Request GET được trình duyệt tự động tạo và gửi đi ngay khi người dùng nhập URL hoặc tải lại trang web để lấy tài nguyên tài liệu HTML.

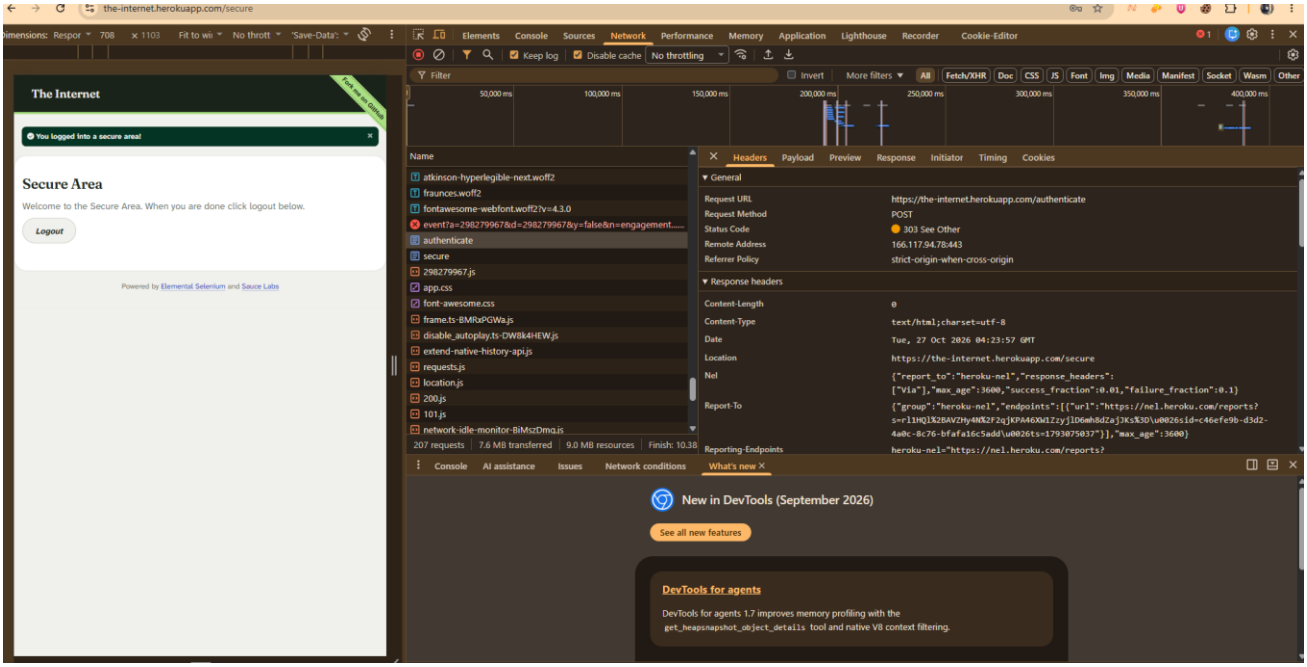


Hình 2: Chi tiết Headers, URL và Response Code của Request GET /login trên DevTools

Thông tin phân tích	Giá trị ghi nhận thực tế
Request URL & Method	GET https://the-internet.herokuapp.com/login
Mã trạng thái (Status Code)	200 OK (Thành công - Máy chủ phản hồi nội dung tài liệu HTML)
Loại tài nguyên (Resource Type)	document (MIME-Type: text/html; charset=utf-8) | Dung lượng: 2.5 KB
Key Response Headers	• Server: Heroku • Content-Type: text/html;charset=utf-8 • X-Frame-Options: SAMEORIGIN (Chống Clickjacking) • X-XSS-Protection: 1; mode=block • X-Content-Type-Options: nosniff (Chống MIME-sniffing)
Key Request Headers	• Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8 • User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) Chrome/153.0 • Upgrade-Insecure-Requests: 1 (Yêu cầu nâng cấp HTTPS)

2.2. Phân tích Request loại POST (Xác thực thông tin tài khoản)

Request POST được kích hoạt khi người dùng bấm nút 'Login' để gửi thông tin đăng nhập từ biểu mẫu (Form) lên máy chủ xử lý.



Hình 3: Chi tiết Request POST /authenticate, Form Payload và Redirect 303 See Other

Thông tin phân tích	Giá trị ghi nhận thực tế
Request URL & Method	POST https://the-internet.herokuapp.com/authenticate
Mã trạng thái (Status Code)	303 See Other (Chuyển hướng thành công sau khi xác thực hợp lệ)
Dữ liệu Payload (Body)	username=tomsmith&password=SuperSecretPassword%21 (Mã hóa dạng URL-encoded)
Header điều hướng (Location)	Location: https://the-internet.herokuapp.com/secure (Chuyển tiếp sang trang quản trị)
Key Response Headers	• Server: Heroku • Content-Length: 0 • Via: 2.0 heroku-router • Location: https://the-internet.herokuapp.com/secure
Key Request Headers	• Content-Type: application/x-www-form-urlencoded • Origin: https://the-internet.herokuapp.com • Referer: https://the-internet.herokuapp.com/login • User-Agent: Chrome/153.0 (Client Web Browser)

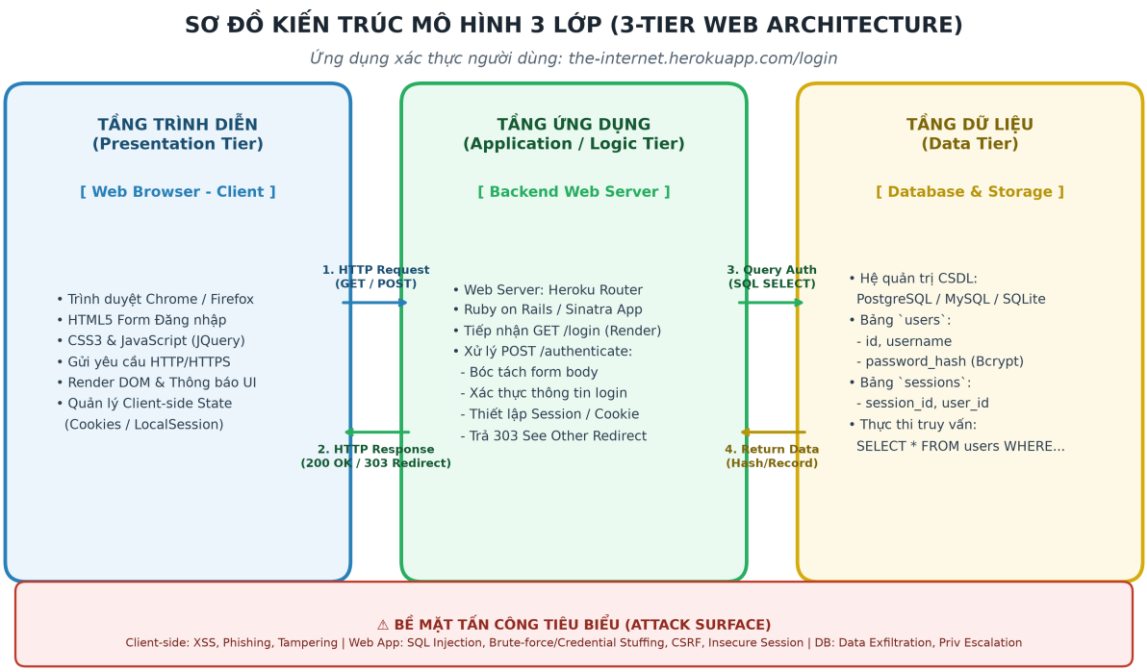
2.3. Bảng phân loại các loại tài nguyên web được tải trong phiên làm việc:

Trong một phiên truy cập trang web, trình duyệt không chỉ tải văn bản HTML đơn thuần mà phải tải rất nhiều thành phần đa phương tiện và kịch bản khác nhau để hoàn thiện giao diện (Rendering Pipeline):

Loại tài nguyên	MIME Type tiêu biểu	Mã trạng thái	Vai trò và ví dụ thực tế
HTML (Document)	text/html	200 OK, 303	Cấu trúc DOM trang web (login, secure)
CSS (Stylesheet)	text/css	200 OK	Định dạng giao diện, màu sắc, bố cục (app.css, sauce.css)
JavaScript (Script)	application/javascript	200 OK	Tương tác logic client-side (jquery, foundation.js)
Image (Ảnh/Vector)	image/svg+xml, png	200 OK	Biểu tượng đồ họa, icon, logo (forkme_right_sauce.svg)
Fonts (Phông chữ)	font/woff2	200 OK	Phông chữ hiển thị văn bản (atkinson, fraunces.woff2)

PHẦN 3: SƠ ĐỒ VÀ MÔ TẢ KIẾN TRÚC MÔ HÌNH 3 LỚP (3-TIER ARCHITECTURE)

Hệ thống ứng dụng web được phân tách thành 3 tầng độc lập theo chuẩn mô hình kinh điển nhằm đảm bảo tính module hóa, khả năng bảo trì và nâng cao năng lực bảo mật:



Hình 4: Sơ đồ kiến trúc mô hình 3 lớp (Client - Server - Database) của hệ thống xác thực web

Mô tả chi tiết chức năng và thành phần của từng tầng:

- **1. Tầng Trình diễn (Presentation Tier / Client):** Bao gồm môi trường trình duyệt web (Google Chrome/Firefox) của người dùng cuối. Tầng này chịu trách nhiệm gửi các yêu cầu HTTP/HTTPS lên máy chủ, nhận dữ liệu HTML/CSS/JS, biên dịch cây DOM và kết xuất giao diện trực quan. Phía Client cũng thực hiện kiểm tra sơ bộ định dạng dữ liệu (Client-side validation) và lưu trữ thông tin phiên tạm thời (Session/Cookies).
- **2. Tầng Ứng dụng & Xử lý nghiệp vụ (Application / Business Logic Tier):** Bao gồm máy chủ Web Server (Heroku Router, Nginx) kết hợp với Backend Application Framework (Ruby on Rails / Sinatra / NodeJS). Tầng

này đóng vai trò 'bộ não' của ứng dụng: tiếp nhận và giải mã HTTP Request, kiểm tra tính hợp lệ của tham số gửi lên, thực hiện quy trình xác thực nghiệp vụ (so khớp hash mật khẩu với cơ sở dữ liệu), khởi tạo Session ID và phản hồi HTTP Response tương ứng (200 OK, 303 Redirect hoặc 401 Unauthorized).

- **3. Tầng Dữ liệu (Data Tier / Database):** Bao gồm hệ quản trị cơ sở dữ liệu (PostgreSQL, MySQL hoặc SQLite). Tầng này có nhiệm vụ lưu trữ bền vững (Persistent Storage) thông tin người dùng, bảng phân quyền tài khoản, bảng nhật ký truy cập (Access Audit Log) và phiên hoạt động. Tầng ứng dụng giao tiếp với tầng dữ liệu thông qua các giao thức nội bộ bảo mật (Database Connection String / ORM / SQL Statements).

PHẦN 4: XÁC ĐỊNH BỀ MẶT TẤN CÔNG (ATTACK SURFACE) VÀ GIẢI PHÁP PHÒNG CHỐNG

Bề mặt tấn công (Attack Surface) là tổng thể tất cả các điểm vào (entry points) mà kẻ tấn công có thể khai thác nhằm xâm nhập trái phép, đánh cắp dữ liệu hoặc phá hoại tính toàn vẹn của ứng dụng. Dựa trên phân tích luồng HTTP và kiến trúc ở các phần trên, các bề mặt tấn công tiêu biểu gồm:

4.1. Bề mặt tấn công tại Biểu mẫu đăng nhập (Login Form / POST /authenticate)

- **Kỹ thuật tấn công dò mật khẩu (Brute-Force & Credential Stuffing):** Kẻ tấn công sử dụng công cụ tự động (như Burp Suite Intruder, Hydra) gửi hàng ngàn request POST chứa danh sách tài khoản/mật khẩu phổ biến nhằm chiếm quyền tài khoản.
  → Giải pháp phòng ngừa: Áp dụng cơ chế Rate Limiting (giới hạn số lượt request/phút theo IP), triển khai CAPTCHA khi đăng nhập sai nhiều lần và kích hoạt xác thực hai yếu tố (2FA/MFA).
- **Tấn công Tiêm mã SQL (SQL Injection - SQLi):** Nếu trường 'username' hoặc 'password' được Backend ghép chuỗi trực tiếp vào câu lệnh truy vấn SQL (ví dụ: SELECT * FROM users WHERE username = '...' AND password = '...'), kẻ tấn công có thể chèn các chuỗi payload như ' OR '1'='1' -- để đăng nhập trái phép mà không cần mật khẩu.
  → Giải pháp phòng ngừa: Luôn sử dụng Parameterized Queries (Prepared Statements) hoặc thư viện ORM, không bao giờ nối chuỗi dữ liệu đầu vào trực tiếp vào câu lệnh SQL.
- **Tấn công Đánh cắp phiên qua trung gian (Man-in-the-Middle - MitM):** Nếu đường truyền không mã hóa HTTPS hoặc cấu hình SSL yếu, kẻ tấn công trên cùng mạng Wi-Fi có thể bắt gói tin chứa plain-text username và password.
  → Giải pháp phòng ngừa: Bắt buộc sử dụng giao thức HTTPS toàn trang, thiết lập Header HSTS (Strict-Transport-Security) để ép trình duyệt luôn kết nối qua kênh mã hóa an toàn.

4.2. Bề mặt tấn công tại Cơ chế quản lý Phiên (Session & Cookie Management)

- **Tấn công Chiếm đoạt phiên làm việc (Session Hijacking):** Nếu Session Token không được bảo vệ bằng các cờ an toàn, hacker có thể đánh cắp token qua mã độc XSS hoặc mạng công cộng để mạo danh người dùng hợp lệ.
  → Giải pháp phòng ngừa: Thiết lập đầy đủ các cờ an toàn cho Cookie: HttpOnly (ngăn JavaScript đọc cookie), Secure (chỉ truyền qua HTTPS) và SameSite=Strict/Lax (chống tấn công CSRF).

4.3. Bề mặt tấn công tại các thành phần mở rộng (Search Box, File Upload, v.v.)

- **Tấn công XSS (Cross-Site Scripting) qua ô tìm kiếm / hiển thị dữ liệu:** Chèn mã kịch bản độc hại <script>...</script> vào các trường nhập liệu để thực thi mã độc trên trình duyệt nạn nhân.
  → Giải pháp: Áp dụng Content Security Policy (CSP), mã hóa ngữ cảnh (Contextual Output Encoding) và kiểm tra lọc dữ liệu đầu vào (Input Sanitization).
- **Tấn công Tải lên tệp độc hại (Malicious File Upload):** Người dùng tải lên shell script (.php, .jsp, .exe) mạo danh tệp ảnh để giành quyền kiểm soát máy chủ Web.
  → Giải pháp: Kiểm tra kỹ định dạng tệp (Magic Bytes), lưu trữ tệp tải lên ở máy chủ tách biệt (như AWS S3) và tắt hoàn toàn quyền thực thi mã nguồn trong thư mục chứa file.

PHẦN 5: NHẬN XÉT CÁ NHÂN VÀ BÀI HỌC KINH NGHIỆM

Sau khi hoàn thành bài thực hành phân tích giao tiếp HTTP và kiến trúc ứng dụng web, em rút ra các bài học và nhận xét cá nhân cốt lõi sau:

- **Về giao tiếp HTTP:** HTTP là giao thức không lưu trạng thái (Stateless), do đó việc duy trì đăng nhập của người dùng hoàn toàn phụ thuộc vào cơ chế Cookie và Session. Nhờ công cụ Developer Tools, luồng dữ liệu trở nên cực kỳ trực quan: từ việc tải HTML ban đầu (GET), gửi thông tin xác thực (POST), nhận tín hiệu điều hướng (303 See Other) cho đến khi nạp đầy đủ các tài nguyên phụ trợ (CSS, JS, Fonts).
- **Về góc độ An toàn thông tin:** Mọi dữ liệu gửi từ phía Client lên máy chủ (bao gồm Header, Body, Cookie, User-Agent) đều có thể bị chỉnh sửa, làm giả hoặc đánh cắp một cách dễ dàng bằng các công cụ chuyên dụng. Do đó, nguyên tắc 'Zero Trust' và 'Never Trust Client Input' phải luôn được áp dụng triệt để ở tầng Backend Logic. Kiểm tra dữ liệu ở phía Client chỉ có giá trị cải thiện trải nghiệm người dùng (UX), hoàn toàn không có giá trị bảo mật độc lập.
- **Về vai trò của Security Headers:** Việc cấu hình chính xác các Response Headers như X-Frame-Options, X-Content-Type-Options, HSTS và Content-Security-Policy đóng vai trò là một lớp phòng thủ chiều sâu (Defense-in-Depth) vô cùng quan trọng giúp bảo vệ ứng dụng trước các cuộc tấn công Clickjacking, MIME-confusion hay XSS.